

Midterm reviewer

Version 2. Each lesson is now a summary first: what the lesson says, in bullets and tables, then its traps, then how the paper examines it, then the code worth recognising. The glossary closes each lesson and is the recall part: cover the right column and say the meaning back.

How to use this. Read a lesson's summary once, then work the glossary with the meaning column covered. The first two lessons are almost pure memory, and the exam has taken 30 items from them, so do not skip them.

Lesson	Summary bullets	Tables	Code boxes	Glossary entries
L01 Introduction to the Ionic framework	10	2	1	29
L01.2 Development environment setup	4	2	2	9
L02 Routing and navigation	18	3	4	16
L03 Angular and Ionic lifecycle	16	1	1	18
L04 Services, and constructor versus ngOnInit	12	2	1	12
L05 Models and interfaces	16	1	1	8
L06 Asynchronous functions	22	1	1	12
L07 Styling, theming and components	10	2	1	15

L01 Introduction to the Ionic framework

Frames the whole course: where mobile app development came from, the four ways an app can be built, what Ionic-Angular is assembled from, and the toolchain that has to exist before the first project. It contains almost no code, and it is the deck the previous written quiz drew from most heavily.

In brief

- **Why cross-platform frameworks exist, and the timeline.** The problem was not only coding. Maintaining separate iOS and Android codebases duplicated the work, doubled the testing, and doubled the distribution effort.
The deck dates the industry in five phases: 2007 (smartphone era, touchscreen phones made apps a daily computing interface), 2008 (app stores, the App Store launched with 500 apps and the Android Market was announced), 2009 to 2012 (hybrid apps, PhoneGap/Cordova put HTML, CSS and JS inside a native wrapper), 2015+ (PWA and cross-platform, offline-ready web apps and one-codebase frameworks matured), 2019+ (Ionic evolves, Ionic 4 moved from Angular-only to Web Components).
- **What Ionic-Angular is assembled from.** Three parts, each with a distinct job. Ionic supplies the mobile UI. Angular supplies the application architecture. Capacitor supplies the bridge to the device.
The runtime is a four-layer stack: your Angular app (pages, services, routes), the Ionic UI layer (mobile components and gestures), Capacitor (native runtime and plugins), and the native SDKs underneath (iOS and Android features).
- **Core building blocks.** Five blocks, and they map directly onto the eight lessons: components (L07), pages and routing (L02), services (L04), plugins (L01, and the device side generally), and theming (L07).
Plugins are Capacitor APIs exposed to JavaScript. The deck names Camera, Filesystem, Geolocation and Storage. They are only reachable inside a native or Capacitor-served build, not on a plain browser preview.
- **Toolchain, CLI workflow and project structure.** Five tools before the first project: Node.js LTS with npm (runs the tooling and installs packages), the Ionic CLI (scaffolds projects and runs commands), Git plus an editor, Android Studio with a JDK (SDK, emulator and native builds), and Xcode on macOS (required to build and sign iOS apps).
The verification commands come before any project is created: `node -v, npm -v, ionic -v, git --version`.
- **Setup failures and the testing ladder.** The deck lists five failure classes with their causes: command not found (PATH, or the terminal needs reopening after an install), build fails (Node version, and only then delete `node_modules` and reinstall), Android errors (missing SDK packages, unaccepted licences, no device detected), iOS build issues (needs macOS, Xcode, a correct bundle identifier, a signing team, provisioning profiles), and a native plugin not working (install the plugin, run `ionic cap sync`, then test on real hardware when the hardware is required).
The recommended order of testing environments, from cheapest to final, is the examinable line: browser, responsive preview, emulator, real device, signed release build.

The four approaches

The four approaches	What it gives	What it costs
Native	Full access to platform APIs, best performance	A separate codebase per platform, so testing and release work double
Web / PWA	One codebase, instant updates, no app store needed	Limited access to device features
Hybrid	One web codebase inside a native container	Wrapper limits; features arrive through plugins
Cross-platform (Ionic)	One web codebase with native access through plugins	Careful performance, plugin and platform testing

The stack, layer by layer

The stack, layer by layer	What it does
Angular	Structure: components, routing, services, the build
Ionic UI components	The mobile look and gestures, as web components
Capacitor	The native runtime and plugin layer bridging web code to device features
Native SDKs	The platform APIs the plugins call: camera, filesystem, geolocation, storage

Traps

- This deck is almost all prose and images. It supplies vocabulary and dates, not code. Do not spend drill time here beyond the CLI commands and the project layout.
- The Ionic CLI command `ionic generate page pages/login` is the fastest way to create a page correctly. Use it rather than creating the four page files by hand under exam pressure.
- `ionic serve` gives a browser preview with live reload. Capacitor plugins generally do not work there, which is why the deck's ladder ends at a real device.

How the paper examines it

Expect identification items here, not code items. Dates (2007, 2008, 2009 to 2012, 2015+, 2019+), the four approaches and their trade-offs, the three parts of Ionic-Angular, the five building blocks, the CLI commands, and the testing ladder. Marked as small print in the companion reviewer.

Code to recognise

THE FOUR CLI COMMANDS THE PAPER ASKS FOR

```
ionic start <name> <template> --type=angular
ionic serve      # runs the app in a browser, live reload
ionic build      # compiles the production web assets
ionic cap sync    # copies the web build into the native project
```

Glossary: L01

Term	What it means, as the exam states it
Capacitor <i>small print</i>	The native runtime and plugin layer in the Ionic-Angular stack that bridges a web app to iOS and Android device features.
PhoneGap <i>small print</i>	The hybrid app wrapper that, between 2009 and 2012, popularised HTML, CSS and JavaScript inside a native container.
ionic start <i>small print</i>	The Ionic CLI subcommand that scaffolds a new project from a name, a template such as tabs, and a type.
ionic serve <i>small print</i>	The Ionic CLI command that runs the app in a browser with live reload.
ionic build <i>small print</i>	The Ionic CLI command that compiles the production web assets.
ionic cap sync <i>small print</i>	The Ionic CLI command that copies the web build into the native platform project and applies plugin changes.
2007 <i>small print</i>	The year the smartphone era began and touchscreen phones made apps a daily computing interface.
2008 <i>small print</i>	The year Apple's App Store launched with 500 apps and the Android Market was announced.
PWA and cross-platform <i>small print</i>	The 2015+ phase in which offline-ready web apps and one-codebase cross-platform frameworks matured.
Ionic 4 <i>small print</i>	The Ionic release, dated 2019 and later, that shifted the framework from Angular-only to Web Components.
Native <i>small print</i>	The development approach with the best access to platform APIs but separate codebases per platform.
Web / PWA <i>small print</i>	The approach that runs in a browser, is the easiest to distribute, and has the most limited native store integration.
Hybrid <i>small print</i>	The approach in which a web app runs inside a native container and is therefore deployable to app stores.
one web-based codebase <i>small print</i>	What cross-platform mobile frameworks were created to reuse: one web-based codebase, keeping mobile-style UI and native device access.
careful performance, plugin and platform testing <i>small print</i>	The typical trade-off of the cross-platform approach: faster development and shared code versus this.
Ionic UI components, Angular, Capacitor <i>small print</i>	The three parts that Ionic-Angular combines.
Angular App, Ionic UI, Capacitor, Native SDKs <i>small print</i>	The four layers through which an Ionic-Angular app runs, from the app down to the device features.
Components, Pages and routing, Services, Plugins, Theming <i>small print</i>	The five core building blocks named in the introduction deck.
Camera, Filesystem, Geolocation, Storage <i>small print</i>	The four Capacitor plugin examples named in the introduction deck.
browser, responsive preview, emulator, real	The recommended progression of testing environments, from the fastest to the final artefact.

Term	What it means, as the exam states it
device, signed release build <i>small print</i>	
declarations <i>small print</i>	The @NgModule metadata array that lists the components, directives and pipes a module owns.
Ionic-Angular	The stack this course uses.
Ionic UI components	The buttons, lists and page shell that look native.
Angular	The framework that structures the app.
Components	Prebuilt mobile UI pieces.
Pages and routing	One component per screen, connected by routes.
Services	Where the logic lives instead of the page.
Plugins	Device hardware reached from web code.
Theming	One central place to change the look.

L01.2 Development environment setup

A click-through of the installations: Node.js, the Ionic CLI, the editor, Android Studio and the SDK. The deck is built from 34 screenshots of installer dialogs, so it contains no code listings at all. It is examined as a sequence of tool names and one verification command list.

In brief

- **The four mandatory installations. Node.js LTS and npm.** Install the LTS release line, not Current. LTS is what the deck calls Recommended For Most Users; Current carries the newest features and the newest breakage. npm ships with Node.
The Ionic CLI: `npm install -g @ionic/cli`. Global, so the ionic command exists in any directory.
- **Verification and environment variables.** Verify all four tools before creating anything: `node -v`, `npm -v`, `ionic -v`, `git --version`. If any one fails, fix it before going further; a missing CLI masquerades as a broken project.
`ANDROID_SDK_ROOT` is the environment variable that tells Android Studio where the Android SDK actually lives.

The toolchain

The toolchain	What it is for	How you know it is ready
Node.js LTS and npm	Runs the CLI and the build	<code>node -v</code> , <code>npm -v</code>
Ionic CLI	Scaffold, serve, build, sync	<code>ionic -v</code>
Git	Version control	<code>git --version</code>
VS Code	The editor for this course	the project folder opens in it
Android Studio and the Android SDK	The Android build and the emulator	Android Studio shows the SDK path
Emulator (AVD)	A virtual Android device	the device appears in the device list
<code>ANDROID_SDK_ROOT</code>	Points Android Studio at the SDK	the path resolves

The run targets, in the order to use them

The run targets, in the order to use them	What it proves
Browser	It works at all: fastest loop, live reload
Responsive preview	The layout at phone size
Emulator	It works on Android, without a phone
Real device	It works on real hardware
Signed release build	It is distributable

Traps

- No code in this deck, so no code can be tested from it. Do not mine it for listings.
- The install order matters: Node before the Ionic CLI, and the SDK plus licences before any Android build.
- The Common Setup Issues list in L01 slide 16 covers most of what actually goes wrong. Read the symptom, not the fix.

How the paper examines it

Tool names and one command list: Node.js LTS and npm, the Ionic CLI, VS Code, Android Studio with a JDK, Xcode for iOS, the four verification commands, `ANDROID_SDK_ROOT`, AVD, and the rule that browser testing precedes native builds. Marked as small print in the companion reviewer.

Code to recognise

VERIFY THE TOOLCHAIN BEFORE CREATING THE FIRST APP

```
node -v
npm -v
ionic -v
git --version
```

POINT ANDROID STUDIO AT THE SDK

```
ANDROID_SDK_ROOT=C:\Users\<you>\AppData\Local\Android\Sdk
```

Glossary: L01.2

Term	What it means, as the exam states it
Node.js LTS and npm small print	The prerequisite toolchain that runs the Angular and Ionic tooling and installs packages.
Ionic CLI small print	The tool that scaffolds projects and runs Ionic commands from the terminal.
Xcode small print	The tool required to build and sign iOS apps, available only on macOS.
node -v, npm -v, ionic -v, git --version small print	The four verification commands given before creating the first Ionic app.
test in the browser first small print	The practice recommended before attempting any native build.
LTS small print	The Node.js release line that must be installed for the Ionic CLI.
Visual Studio Code small print	The text editor recommended for Ionic development in this course.
ANDROID_SDK_ROOT small print	The environment variable that points Android Studio at the location of the Android SDK.
Android Virtual Device small print	The emulated Android device created inside Android Studio so the app can be run without a physical phone.

L02 Routing and navigation

The most heavily weighted lesson, and the one the whole application depends on. It defines the routing vocabulary, then covers template navigation, code navigation, route parameters, back navigation, tab stacks and guards. It is taught entirely in the standalone API, so every listing here has to be re-expressed in NgModule form.

In brief

- **Routing is the map, navigation is the movement.** A route can exist before anybody visits it. Navigation is what uses those routes to change the visible page. The deck separates the two deliberately, and the exam follows.
Ionic replaces Angular's plain `<router-outlet>` with `<ion-router-outlet>`. That single substitution is what gives pages mobile transitions and the navigation stack.
- **The route list is the application map.** Four properties do all the work: `path` (the URL part after the domain), `loadComponent` (loads the page only when needed), `redirectTo` (chooses the first screen), and `pathMatch: 'full'` (redirect only when the path is exactly empty).
The deck states the consequence directly: if the route is missing, the page cannot be opened by URL. That is the first thing to check when a page will not open.
- **Declaring navigation in the template.** `routerLink` is for navigation that is known directly in the HTML. When the destination depends on data, login status or a function result, move to TypeScript navigation.
`routerDirection` controls the stack, not the destination: forward when the user is moving deeper, back when returning to an earlier page, and root when the stack must be reset.
- **Navigating from code: Router and NavController.** Two objects, two jobs. Use Router for general Angular navigation, route parameters and query parameters. Use NavController for Ionic-friendly forward, back and root stack behaviour.
`this.router.navigate(['/products', id])` pushes onto the stack. `this.navCtrl.navigateRoot('/home')` sets a root move instead of a forward push.
- **Route parameters and query parameters.** A route parameter identifies one exact resource: `/products/42`. A query parameter modifies how a list is shown: `/products?category=books`. Route parameters are for required identity, query parameters for optional filters.
Reading a route parameter is a two-step job: get the parameter map from the activated route, then read the named key. The deck's snippet uses `this.route.snapshot.paramMap.get('id')`.
- **Back navigation and the stack.** `ion-back-button` follows Ionic's navigation history rather than the browser's. It goes into the toolbar's start slot.
`defaultHref` is the fallback destination used when there is no previous page, which is exactly the case when a URL is opened directly or the page is reloaded.
- **Tabs mean several stacks.** Linear navigation has one path forward and backward. Tab navigation is non-linear: each tab keeps its own history stack, so Back is not always the previously seen screen.
The deck's important warning is examinable in both directions: with tabs, do not assume browser back equals app back.
- **Guards protect the route, not the button.** A guard runs before the route opens. Returning true allows navigation; returning a `UrlTree` redirects safely.
The deck's framing is the examinable point: never rely only on hiding buttons. Protect the route itself, or the page is still reachable by typing its URL.
- **Debugging routing problems.** The checklist, in the deck's order: check the browser URL, check the route array for a misspelled path, check that the wildcard is last, check imports and page filenames, and check the console for 'Cannot match any routes'.
The console-log pattern before `Router.navigate()` is worth keeping: it separates 'the function never ran' from 'the function ran and the route is wrong'.

The route map

The route map	Path	Opens
app-routing.module.ts	''	redirect to /login, pathMatch: 'full'
app-routing.module.ts	'login', 'tabs'	the two page modules, by loadChildren
tabs-routing.module.ts	'movies', 'menu',	the four tab modules

The route map	Path	Opens
	'search', 'account'	
tabs-routing.module.ts	''	redirect to /tabs/movies
movies-routing.module.ts	'' and ':id'	MoviesPage, then MovieDetailPageModule: /tabs/movies/01
movie-detail-routing.module.ts	''	MovieDetailPage, empty because the parent took ':id'

What you write, and where

What you write, and where	How
Navigate from a template	routerLink on the element
Navigate from code	Router.navigate([...])
Reset the stack after logout	NavController.navigateRoot(url, { replaceUrl: true })
Back, with a fallback	ion-back-button with defaultHref
Keep a page out of reach	a canActivate guard

Two kinds of parameter

Two kinds of parameter	Route parameter	Query parameter
Where	a path segment, :id	after the question mark
Use it for	identity: which record	options: how the list is shown
Reading it	route.snapshot.paramMap.get('id')	route.snapshot.queryParamMap.get('filter')
Type	always a string	always a string

Traps

- Every listing in this deck is standalone. `app.routes.ts` must become `app-routing.module.ts`, `loadComponent` must become `loadChildren`, and the top-level children: `[]` of the tabs route must move into `tabs-routing.module.ts`. Copying a deck listing verbatim will not work here.
- `pathMatch: 'full'` is required on every empty-path redirect, including the child redirect inside the tabs shell. Without it the redirect matches every URL under the parent.
- The wildcard `**` must be the last route. Place it before the others and every route below it becomes unreachable.
- Route parameters are strings. Convert before comparing.
- For a nested detail route, `:id` is declared once, in the catalogue's own routing module. Repeating it in the detail module produces an unreachable route.

How the paper examines it

The most weighted lesson for both halves. Identification: route, path, outlet, routerLink, stack, routerDirection values, route versus query parameter, defaultHref, guard. Code: writing a routing module in NgModule form, and reading a route parameter in the right hook with the right conversion.

NgModule note: This is the lesson that must be translated rather than quoted. The full NgModule translation is in section 2, and the working version is in `app-code/`: one `app-routing.module.ts` with a single `forRoot`, `loadChildren` for every lazy page, and one `x-routing.module.ts` per page using `forChild`.

Code to recognise

THE ROUTE MAP: THE ONE FILE THAT CALLS FORROOT

```

import { NgModule } from '@angular/core';
import { PreloadAllModules, RouterModule, Routes } from '@angular/router';

const routes: Routes = [
  {
    path: '',
    redirectTo: 'login',
    pathMatch: 'full',
  },
  {
    path: 'login',
    loadChildren: () => import('./login/login.module').then(m => m.LoginPageModule),
  },
  {
    path: 'tabs',
    loadChildren: () => import('./tabs/tabs.module').then(m => m.TabsPageModule),
  },
];

@NgModule({
  imports: [RouterModule.forRoot(routes, { preloadingStrategy: PreloadAllModules })],
  exports: [RouterModule],
})
export class AppRoutingModule {}

```

THE TABS SHELL: CHILDREN, EACH LAZILY LOADED

```

import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { TabsPage } from './tabs.page';

const routes: Routes = [
  {
    path: '',
    component: TabsPage,
    children: [
      {
        path: 'movies',
        loadChildren: () => import('../movies/movies.module').then(m => m.MoviesPageModule),
      },
      {
        path: 'menu',
        loadChildren: () => import('../menu/menu.module').then(m => m.MenuPageModule),
      },
      {
        path: 'search',
        loadChildren: () => import('../search/search.module').then(m => m.SearchPageModule),
      },
      {
        path: 'account',
        loadChildren: () => import('../account/account.module').then(m => m.AccountPageModule),
      },
      {
        path: '',
        redirectTo: '/tabs/movies',
        pathMatch: 'full',
      },
    ],
  },
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule],
})
export class TabsPageRoutingModule {}

```

A CATALOGUE OWNING ITS OWN DETAIL ROUTE

```

import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { MoviesPage } from './movies.page';

const routes: Routes = [
  {
    path: '',
    component: MoviesPage,
  },
  {
    // the catalogue owns its own detail segment, so the URL is /tabs/movies/01
    path: ':id',
    loadChildren: () =>
      import('../movie-detail/movie-detail.module').then(m => m.MovieDetailPageModule),
  },
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule],
})
export class MoviesPageRoutingModule {}

```

THE DETAIL MODULE'S PATH IS THE EMPTY STRING

```

import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';

import { MovieDetailPage } from './movie-detail.page';

const routes: Routes = [
  {
    // the parent route already carried /movies/:id, so this module sits at ''
    path: '',
    component: MovieDetailPage,
  },
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule],
})
export class MovieDetailPageRoutingModule {}

```

Glossary: L02

Term	What it means, as the exam states it
ion-router-outlet	The Ionic component that replaces plain router-outlet so routed pages receive mobile transitions and a navigation stack.
route	The word for a rule that maps a URL path to a component or a lazily loaded page.
path	The URL segment after the domain, such as home or products/:id.
outlet	The placeholder in a template where the matched page is displayed.
routerLink	The HTML attribute used for tap or click navigation from a template.
Router.navigate()	The TypeScript function called on the router to navigate from code.
stack	The mobile-style history of pages that Ionic keeps so back behaviour works.
pathMatch: 'full'	The route option that makes an empty path match only the exactly empty URL, rather than every URL.
	The wildcard path used as the last route to catch an unknown URL and redirect it somewhere safe.
query parameter	The kind of parameter that modifies how a list is shown, appears after a question mark, and is best for optional filters.
root	The routerDirection value that resets the navigation stack rather than moving forward or back.
ion-back-button and defaultHref	The Ionic component that provides navigation-history back behaviour, together with the attribute that supplies a fallback destination when the history is empty.
loadChildren	The route property that lazily loads a module in an NgModule project, as opposed to the standalone form that loads a component class.
route guard	The Angular mechanism that runs a check before a route opens, so that a login-only page cannot be reached by typing its URL.

Term	What it means, as the exam states it
forRoot	The RouterModule static method called exactly once in the application, in the application routing module.
forChild	The RouterModule static method every page routing module uses, and never the root method.

L03 Angular and Ionic lifecycle

Why code that runs at the wrong moment behaves strangely. It separates Angular's component lifecycle from Ionic's page lifecycle, gives a decision guide for where each kind of code belongs, and explains why a page can leave the screen and still be alive. The examinable core is a short list of hooks and their correct uses.

In brief

- **Two lifecycles, and the confusion they cause.** Angular's lifecycle is about components being created, checked, rendered and destroyed. Ionic's is about pages entering and leaving mobile navigation.
The whole difficulty in one sentence: in Ionic, a page can leave the screen but still remain alive in memory.
- **Angular lifecycle hooks.** Five hooks. The first two carry almost all of the examinable weight.
constructor runs first, when the class is created, and is for dependency injection. `ngOnInit()` runs once after Angular initialises inputs, and is for one-time setup.
- **Ionic page lifecycle hooks.** Four hooks bracket visibility: `ionViewWillEnter()` before the page enters, `ionViewDidEnter()` after it has finished entering, `ionViewWillLeave()` before it leaves, and `ionViewDidLeave()` after it has left.
The correct uses are the examinable part. Will-enter is for refreshing data before the user sees it. Did-enter is for work that needs the page visible, such as focus, animation-sensitive work or plugin UI. Will-leave is for pausing timers, stopping the camera, and unsubscribing. Did-leave is for logic after the transition.
- **Entry and exit sequences.** First-time entry runs in a fixed order: constructor (class created), `ngOnInit` (init once), `ionViewWillEnter` (about to show), the transition animation, then `ionViewDidEnter` (now visible).
Re-entry runs a shorter sequence. `ngOnInit` may not repeat, but the Ionic enter hooks do.
- **The decision guide, and the memory phrase.** This slide is the one to memorise, because it answers the practical question directly: constructor to inject services only, `ngOnInit` for one-time setup and data that does not need refreshing, `ionViewWillEnter` to refresh on every visit, `ionViewDidEnter` for work that needs a visible page, `ionViewWillLeave` to pause or stop things, `ngOnDestroy` for final cleanup when the component is actually destroyed.
The memory phrase compresses it correctly: constructor receives the tools, `ngOnInit` prepares once, `ionViewWillEnter` refreshes each visit, `ionViewWillLeave` pauses or cleans up when leaving.
- **Worked examples: load once, refresh each visit, stop on leave. Example 1, load once.** A profile page injects the service in the constructor and loads the data in `ngOnInit`. Correct because profile data rarely changes during a session. The deck's stated reason it is not in the constructor: the constructor should not do heavy work, Angular inputs may not be ready there, and a simple constructor is easier to debug.
Example 2, refresh each visit. An orders page loads in `ionViewWillEnter` because the user may add, edit or delete data on another page and the list must be current on return. The deck's scenario is exactly the tabs flow: orders list, add order, back to orders.
- **Common mistakes and debugging.** Six mistakes with their fixes: everything in `ngOnInit` (data does not refresh on a cached page, move it to `ionViewWillEnter`), heavy work in the constructor (inject only, then set up in a hook), waiting for `ngOnDestroy` (Ionic may hide without destroying, so pause in `ionViewWillLeave`), touching the DOM or a plugin too early (use `ionViewDidEnter` or `ngAfterViewInit`), no logging (stop guessing the order), and the wrong component (Ionic hooks only run on routed page components, so child components must use Angular hooks).
The debug checklist is a five-step procedure: open the console, navigate A to B and back to A, observe which hooks repeat, set breakpoints inside the hooks, and use the Network tab to confirm when requests really fire.
- **Quick formula and summary.** The four-line formula is the examinable compression of the whole lesson: need a service, use the constructor; need one-time setup, use `ngOnInit`; need fresh data every visit, use `ionViewWillEnter`; need to stop something when leaving, use `ionViewWillLeave`.
The summary restates the frame: lifecycle hooks are automatic functions triggered by Angular or Ionic, Angular's manage creation and destruction, Ionic's manage entering and leaving, and because pages can be hidden but alive, refresh and cleanup logic must be placed deliberately.

The hooks, in the order they run

The hooks, in the order they run	When	Use it for
----------------------------------	------	------------

The hooks, in the order they run	When	Use it for
constructor	creating the object	wiring dependencies only
ngOnInit	once, after creation	the first data load
ionViewWillEnter	before the page shows, on every entry	refresh-on-return
ionViewDidEnter	after the entry animation	work that needs the page visible
ionViewWillLeave	before leaving	pause timers, stop the camera, unsubscribe
ionViewDidLeave	after leaving	cleanup that needs the page gone
ngOnDestroy	only when the page is removed	final teardown

Traps

- Do not put data loading in the constructor. The deck states it as a rule, the drill enforces it, and it is the most likely true-or-false trap in this lesson.
- Do not expect ngOnDestroy when navigating away with tabs. The page is hidden, not destroyed.
- Ionic hooks do not run on child components. If a small component inside a page needs to react, it must use Angular hooks.
- The deck's code is already class-based, so it ports to NgModule form almost unchanged. The one addition in this project is `standalone: false` on the component decorator, required on Angular 19 and later because these pages are declared in modules.

How the paper examines it

Identification: the five Angular hooks, the four Ionic hooks, page caching. Short answer and true-or-false: when to load once versus refresh every visit, and why ngOnDestroy is the wrong place for leave-time cleanup. Code: writing a page class that loads in the right hook.

NgModule note: Almost no translation needed. The hooks are identical in both APIs. The additions for this project are `standalone: false` on the decorator and `implements OnInit` kept as-is.

Code to recognise

EVERY HOOK YOU MUST BE ABLE TO NAME, IN THE ORDER IT RUNS

```
// 1. Wiring only: create the object, store injected dependencies.
constructor(private movieService: MovieService) {}

// 2. Runs once, after creation. First place page data may be loaded.
ngOnInit(): void { this.load(); }

// 3. Ionic only, routed pages only: every time the page is about to show.
ionViewWillEnter(): void { this.load(); }

// 4. About to leave: pause timers, stop the camera, unsubscribe.
ionViewWillLeave(): void { this.stopTimer(); }

// 5. Final teardown. Ionic calls it only when the page is really removed.
ngOnDestroy(): void { this.subscription.unsubscribe(); }
```

Glossary: L03

Term	What it means, as the exam states it
constructor	The TypeScript class member that runs first and exists to create the object and store injected dependencies.
ngOnInit	The Angular lifecycle hook that runs once, after the component is created, and is the first place page data may be loaded.
ionViewWillEnter	The Ionic hook that runs when a page is about to enter the visible screen, and is the correct place for refresh-on-return logic.
ionViewDidEnter	The Ionic hook that runs after the entry animation, when the page is now visible.
ionViewWillLeave	The Ionic hook that runs when a page is about to leave, and is the correct place to pause timers, stop the camera, or unsubscribe.
ionViewDidLeave	The Ionic hook that runs after the page has finished leaving the screen.
ngOnDestroy	The Angular hook that performs final teardown, and which Ionic calls only when the page is actually removed rather than hidden.

Term	What it means, as the exam states it
page caching	The behaviour that explains why <code>ngOnDestroy</code> does not run every time you navigate away, and why page state and scroll position survive a return.
routed page components only	The restriction that decides which components receive the Ionic navigation hooks.
constructor, <code>ngOnInit</code> , <code>ionViewWillEnter</code> , <code>ionViewDidEnter</code>	The four hooks, in order, that a routed page passes through on its first entry.
<code>ngOnInit()</code>	Prepare once.
<code>ngOnChanges()</code>	React to a parent's new value.
<code>ngAfterViewInit()</code>	The template now exists; touch it here.
<code>ngOnDestroy()</code>	Last chance to release something.
<code>ionViewWillEnter()</code>	Refresh on every visit.
<code>ionViewDidEnter()</code>	The page is on screen; now act.
<code>ionViewWillLeave()</code>	Stop the background work.
<code>ionViewDidLeave()</code>	Anything left to do once hidden.

L04 Services, and constructor versus ngOnInit

Where code belongs. It defines what a service is, how dependency injection hands it to a page, and then draws the line that most beginners cross: the constructor creates the object, `ngOnInit` starts the page, and reusable logic belongs in the service. The examinable core is that three-way split.

In brief

- **What a service is, and the duplication it removes.** A service is a TypeScript class used to share logic. Angular injects it into the components and pages that need it, rather than each page building its own copy.
The deck draws the before-and-after plainly. Before services, each page repeats the same API call, storage access and business rules. With services, one reusable class holds the logic and every page uses it.
- **Dependency injection and instance sharing.** Angular gives the service to the component through dependency injection. The page declares what it needs in the constructor and never constructs the service itself with `new`.
`providedIn: 'root'` tells Angular to create one shared application-wide instance, with no entry needed in any module's providers array. That is why a service registered this way can hold shared state across pages.
- **The three homes for code: service, constructor, ngOnInit.** This decision guide is the most examinable table in the lesson. Three destinations, three different justifications.
Constructor: dependency injection, very light default values, no visible side effects. It is about creating the object.
- **Explicit service initialisation.** Startup work should be explicit rather than hidden in a service constructor. The deck's pattern is a named method such as `service.initSession()` called from `AppComponent`.
The companion anti-pattern is a service that quietly does work when it is first injected, so nothing in the code says when it happened. The deck calls for explicit methods such as `getData()`, `initSession()` and `addNote()`.
- **The three worked service examples. Example 1, simple shared data.** A plain array held in the service and read by pages. No network, no storage. This is the shape the canonical project uses for its catalogues.
Example 2, storage-style settings. The service reads and writes persisted values, so the pages never touch storage directly.
- **Debugging and the three mistakes.** Two checks solve most service failures: confirm the service is registered, either through `providedIn: 'root'` or through an entry in the module's providers array; and read the browser console and the Network tab for the actual error.
The three mistakes follow the same theme: logic in the wrong home. Code that belongs in the service is copied into pages until the pages disagree with each other; data loading sits in the constructor; and startup work hides inside the service.

The constructor against ngOnInit

The constructor against ngOnInit	`constructor`	`ngOnInit`
Job	create the object, store injected dependencies	the first place page data may be touched
Contains	injected dependencies only	loading, service calls
Never	API calls, business rules, work that can fail	keep it about data

Two ways to register a service

Two ways to register a service	Where it is written	Instances
<code>providedIn: 'root'</code>	on the decorator	one, app-wide (this project)
<code>providers: [Service]</code>	a module's providers array	one per injector that provides it

Traps

- A service has no lifecycle hooks. `ngOnInit` on a service does nothing. If startup work is needed, expose a named method and call it explicitly.
- The constructor is for dependency injection. API calls, alerts and navigation in the constructor are the lesson's named wrong pattern.
- `@Injectable({ providedIn: 'root' })` needs no providers entry. A bare `@Injectable()` does.
- There is no `@Service` decorator in Angular. The reference activity in this workspace uses one in two files and will not compile.

How the paper examines it

High value for both formats. Identification: service, dependency injection, providedIn: 'root'. Short answer and true-or-false: what belongs in the constructor versus ngOnInit versus a service method, and why. Code: writing a service with providedIn: 'root' and a page that injects it and loads in ngOnInit.

NgModule note: One addition only. providedIn: 'root' works identically in both APIs, so no translation is needed for a service. If the alternative registration form is used instead, the entry goes in the declaring module's providers: [] array, which is NgModule-specific.

Code to recognise

THE TWO REGISTRATION FORMS, AND WHERE THE WORK GOES

```
// Form 1: on the decorator. One shared instance, no providers entry.
@Injectable({ providedIn: 'root' })
export class MovieService {
  private movies: Movie[] = [];
}

// Form 2: in a module's providers array.
providers: [MovieService]

// Constructor: wiring only, never work.
constructor(private movieService: MovieService) {}

// ngOnInit: the work.
ngOnInit(): void { this.movies = this.movieService.getMovies(); }
```

Glossary: L04

Term	What it means, as the exam states it
dependency injection	The Angular mechanism by which a service is handed to the component or page that needs it, rather than the page constructing it.
providedIn: 'root'	The service registration form that means Angular creates one shared app-wide instance with no providers array entry.
dependency injection	The correct contents of a component constructor, in three words or fewer.
load initial page data	The correct contents of ngOnInit in a page.
a service method	The correct home for API calls, storage logic, business rules and reusable app logic.
calling APIs, showing alerts, navigating	The three things a constructor must never do, named in the deck's own words.
service.initSession()	The explicit call made from AppComponent when service startup work must happen at app launch instead of hiding in the service constructor.
whether it is registered	The first thing to check in the browser when a service throws a NullInjectorError at page load.
Service	One reusable class instead of copies per page.
Constructor	Receive your tools.
ngOnInit	Prepare the page once.
Service method	Anything more than one page might need.

L05 Models and interfaces

How to describe the shape of application data so the compiler can catch mistakes before they reach the template. It contrasts the interface (a compile-time contract, no runtime code) with the model class (a blueprint that creates objects and carries behaviour), and gives a simple rule for choosing.

In brief

- **Why structured data matters.** Without structure, data can be missing, wrong property names are hard to catch, the code becomes guesswork, and templates break silently.
With an interface or model, the expected fields are clear, TypeScript warns early, autocomplete becomes useful, and both service and component code get cleaner.
- **The three basic terms.** An **object** is a group of related values: one student, one product, one task.
An **interface** is a TypeScript contract. It says what properties an object should have and it has no runtime code. It disappears when the application runs.
- **Interfaces: the data shape.** An interface lists fields and their types. `id`, `name`, `price` and `isAvailable` must exist; `imageUrl?: string` may be missing. The question mark is the optional marker.
Interfaces are compile-time only. They help TypeScript check the code before it runs, and contribute nothing to the shipped JavaScript.
- **Model classes: data plus behaviour.** A model class is for when the object itself should know how to do something. The deck's rule: use a class when the object needs logic, default values, or helper methods.
Three features carry the lesson: the constructor creates the object and initialises values; a default value such as `isDone: boolean = false` applies when no value is given; and methods such as `toggleDone()` and `getStatusText()` add behaviour.
- **Interface versus model class.** Three differences decide it. Does it create an object: the interface cannot, the model class does, with `new`. Does it have methods: no, yes. Does it produce runtime code: no, yes.
Best for an interface: an API response shape, simple application data, forms and lists. Best for a model class: default values, helper methods, data with behaviour.
- **Where these files live.** `src/app/models/` holds interfaces and model classes, named for what they describe such as `student.interface.ts` or `todo.model.ts`.
`src/app/services/` fetches, saves, updates or transforms data. `src/app/pages/` uses the data and shows it through Ionic UI. `src/app/shared/components/` holds reusable pieces.
- **Interfaces with forms and with services.** A form object typed by an interface is not random: the expected input fields are documented, and TypeScript catches wrong field names. The deck's register form is the pattern, bound with `[(ngModel)]`.
The three-part flow through a service is the shape the canonical project uses: the interface defines the shape, the service requests or returns a typed list, and the page receives `Type[]` and displays it safely.
- **Interface versus type alias, and the common mistakes.** Both describe data. Use an interface for common object shapes such as `Student`, `Product` or `User`. Use a type alias for union values, shortcuts, or advanced combinations, as in `type UserRole = 'admin' | 'student' | 'guest'`.
Three mistakes with their fixes: typing everything as `any` (TypeScript cannot help, so use `Product[]`, `Student[]` and similar), wrong property names (`product.title` instead of `product.name`, so match the interface fields), and expecting an API response to arrive as a class instance.

Interface against class

Interface against class	`interface`	`class`
Exists at runtime	no	yes
Creates objects with <code>new</code>	no	yes
Holds methods	no	yes
Use it for	describing the shape of data	behaviour on data

Traps

- An interface carries no runtime code, so it cannot hold a method implementation. If behaviour is needed, it must be a class or a method in the consuming service.
- The optional marker is a single question mark on the property, not on the type.
- Records that arrive from an API are plain objects. They do not gain model methods unless each one is mapped with `new Model(...)`.
- Use the interface or class that already exists rather than declaring a second shape for the same record; duplicated shapes drift apart and the compiler stops being useful.

How the paper examines it

Identification: interface, model class, the optional marker, the folder convention, the type alias. Short answer and true-or-false: interface versus model class and when to use each; whether an interface exists at runtime. Code: writing an interface, typing an array with it, and writing a model class with a method.

NgModule note: No translation needed. Interfaces and model classes are plain TypeScript and behave identically in both APIs. Nothing about them touches the module system.

Code to recognise

A SHAPE, AND A BLUEPRINT THAT CAN BUILD OBJECTS

```
export interface Movie {
  id: number;
  title: string;
  posterUrl: string;
  genre: string;
  synopsis: string;
  gallery: string[];
}

// The same record as a class: it exists at runtime and can hold methods.
export class MovieModel {
  constructor(
    public id: number,
    public title: string,
    public posterUrl?: string
  ) {}

  rating(): string { return this.id > 2 ? 'Top pick' : 'Also showing'; }
}
```

Glossary: L05

Term	What it means, as the exam states it
interface	The TypeScript construct that states what properties an object should have, carries no runtime code, and exists only to check code before it runs.
model class	The construct that acts as a blueprint able to create objects with <code>new</code> , may include a constructor and methods, and exists at runtime.
the question mark	The syntax that marks a property as possibly missing, for example <code>imageUrl?: string</code> .
<code>new</code>	The keyword used to create an object from a model class.
type alias	The TypeScript feature suited to union values and shortcuts rather than to describing an object shape, as in <code>type UserRole = 'admin' 'student'</code> .
map them into model instances	What must be done to plain objects returned by an API before their model methods can be called.
<code>src/app/models</code>	The folder where interfaces and model classes belong in this project layout.
Object	One record.

L06 Asynchronous functions

The largest deck, and the one that explains why an app feels broken when it is only slow. It covers synchronous versus asynchronous execution, Promises, `async` and `await`, error handling with try/catch/finally, the async pipe, choosing between a Promise and an Observable, and the Ionic feedback pattern that keeps the user informed.

In brief

- **Why mobile apps are asynchronous at all.** Mobile apps usually wait for something external: an API or server response, login or token validation, a local storage read or write, a camera, GPS, Bluetooth or filesystem operation, or a navigation and page refresh.
The stated goal is that the app feels smooth because long tasks run asynchronously and the user receives clear feedback.
- **Promises and the three states.** A Promise is a future result with three states: pending, settled, then either fulfilled with a success value or rejected with an error reason.
The practical consequence is that every Promise has two exits, and both must be handled. Code that handles only the fulfilled path leaves the UI in an inconsistent state when the request fails.
- **async, await, and the effect on the rest of the app.** `async` marks a function that is allowed to use `await`. `await` pauses only that function until the Promise settles.
The app itself keeps responding: taps still register while the function is waiting. That distinction is the point of the lesson and a likely true-or-false item.
- **Error handling: try, catch, finally.** `try` holds the `async` task that may fail, `catch` handles the failure (a failed login, a network error), and `finally` performs cleanup such as hiding the loading indicator and re-enabling the button.
The deck explains why `finally` matters specifically in Ionic: UI elements must be dismissed even when an error occurs, or the spinner stays on screen forever.
- **The async pipe and the dollar convention.** The async pipe subscribes to an Observable, updates the template, and unsubscribes automatically when the component is destroyed. That removes the memory leak a forgotten manual `unsubscribe()` causes.
Observable variables take a `$` suffix by convention, as in `products$`. The suffix is a signal to the reader that the value arrives later.
- **Choosing between a Promise and an Observable.** A Promise is one future value, used with `async` and `await`, well suited to a login result, a storage read, or a modal dismissal. It cannot easily be cancelled once started.
An Observable can emit one or many values, is used with the async pipe or RxJS, suits `HttpClient` and streams, and can be unsubscribed or cancelled.
- **The Ionic feedback pattern.** The five-step pattern for any button-triggered `async` action: tap, show loading, await the task, handle the result, dismiss or show a toast.
The design rule behind it is the line worth quoting: Ionic users should never wonder whether the app received their tap.
- **Parallel tasks.** Sequential execution waits for task one, then starts task two. Parallel execution starts all of them together and waits for the group.
`Promise.all` is the parallel tool, and it is safe only when the tasks do not depend on each other. Chaining dependent calls in `parallel` produces races rather than speed.
- **Where async code belongs in a page.** `ngOnInit` for initial setup and a data load that should happen once when the page is created. `ionViewWillEnter` when the data must refresh on every visit, including after returning from another page.
One line is stated as an absolute and is very likely to be examined: never make the constructor `async`.
- **The six common bugs and the debugging checklist.** The six bugs are a checklist in themselves: forgetting `await`, so the code works with the Promise object instead of its result; no `catch` block, so errors only appear in the console; loading never dismissed, so the UI sticks; manual `subscribe` leaks; duplicate taps sending repeated requests; and `async` work in the constructor.
The debugging procedure: open the console, inspect the Network tab for the real API status, log before and after the `await`, log both `error.message` and the full error, and disable the button while loading.
- **Best practices, the activity, and the summary.** Five habits: keep `async` functions small, move API calls into services, use the async pipe for template data, avoid duplicated requests, and always give the user feedback.

The activity's expected behaviour is a compact acceptance test for everything above: the button disables while waiting, a message appears after about 1.5 seconds, no duplicate login request is sent, and the loading state resets even when the request errors.

Promise against Observable

Promise against Observable	`Promise`	`Observable`
Values	one	a stream
Runs	on creation	on subscribe
Cancellation	none	unsubscribe
In a template	await in the class	the <code> async</code> pipe, which unsubscribes for you

Traps

- `finally` is not optional decoration. Clearing a loading flag anywhere else leaves the spinner stuck after a failure.
- Never make the constructor `async`, and never `await` inside it.
- Forgetting `await` is silent: the code keeps running with the Promise object instead of the result, and the bug shows up later as undefined data.
- A manual `subscribe()` must be unsubscribed. The `async` pipe does it automatically, which is why it is preferred in templates.
- `Promise.all` is only for independent tasks.

How the paper examines it

The heaviest lesson after routing for short answers and true-or-false. Identification: Promise states, `await`, `finally`, the `async` pipe, the `$` suffix, `Promise.all`. Short answer: why `finally` is required, Promise versus Observable, and where `async` code belongs. Code: writing an `async` load that sets a flag, awaits a service, catches the failure and clears the flag in `finally`.

NgModule note: No translation needed. Async behaviour, Promises, Observables and the `async` pipe are identical in both APIs.

Code to recognise

AN ASYNC LOAD THAT ALWAYS RESTORES THE UI

```
async load(): Promise<void> {
  this.loading = true;
  this.error = '';
  try {
    this.movies = await this.movieService.getMovies();
  } catch (err) {
    this.error = 'Could not load the catalogue.';
  } finally {
    this.loading = false; // runs whether the task succeeded or failed
  }
}
```

Glossary: L06

Term	What it means, as the exam states it
Promise	The object representing a future result, whose two settled outcomes are fulfilled and rejected.
await	The keyword that pauses the execution of the enclosing function until the promise settles, without freezing the app.
async	The function modifier required before <code>await</code> may be used inside it.
finally	The block that runs whether the awaited task succeeded or failed, and is where the loading indicator is dismissed.
async pipe	The Angular template pipe that subscribes to an Observable, updates the view, and unsubscribes automatically when the component is destroyed.
the dollar suffix	The naming convention applied to Observable variables such as <code>products\$</code> .
Promise.all	The method used to start several independent <code>async</code> tasks at the same time and wait for all of them.
Synchronous	One thing at a time, in order.
Asynchronous	Start it, keep the app alive, handle the answer later.
try	Attempt the risky work.
catch	Decide what the user sees when it fails.

Term	What it means, as the exam states it
Observable	A stream of answers, cancellable.

L07 Styling, theming and components

How to make the app look deliberate rather than default. It gives a four-layer styling model, the three stylesheets and what each is for, the colour-name system and the theme variables that back it, why ordinary CSS selectors cannot reach inside Ionic components, and the component families to reach for.

In brief

- **The four layers, and what each is for.** The layers are Ionic components, theme variables, page and component styles, and behaviour. Together they produce the styled app.
The deck's formula is the examinable line: good UI equals the correct component, plus a consistent theme, plus small custom styles.
- **Colour names and theme variables.** The template asks for a colour by name, and the theme supplies the actual value. That is why changing one variable re-themes every component that used the name.
The eight names and their intended semantics are worth memorising: primary for main brand actions, secondary for supporting actions, tertiary for extra emphasis, success for positive status, warning for caution, danger for delete and error, medium for neutral, and light for light UI.
- **Reaching inside a component: CSS variables and shadow parts.** Ionic components use the Shadow DOM, so an ordinary CSS descendant selector cannot reach their inner elements. This is the single most common styling frustration in the lesson.
Two supported doors exist. Component CSS variables are written in a rule that targets the Ionic component and can be read as 'change the button's public style settings'. Shadow parts use `::part(...)` to style an element the component deliberately exposes.
- **The component families and the layout grid.** Seven families, and choosing correctly is most of the UI work: structure (`ion-header`, `ion-toolbar`, `ion-content`), layout (`ion-grid`, `ion-row`, `ion-col`), display (`ion-card`, `ion-list`, `ion-item`), form (`ion-input`, `ion-select`, `ion-checkbox`), action (`ion-button`, `ion-fab`, `ion-modal`), and status (`ion-chip`, `ion-badge`, `ion-toast`).
The grid is responsive by attribute rather than by media query: `size-md="6"` produces two cards per row on medium screens, while phones show one.
- **The lesson task and what it checks.** The deck's own task is a useful rehearsal for the canonical project: build a page that uses `color="primary"` on a toolbar and a button, customises an `ion-card` through CSS variables, and uses `ion-badge` for status.
The task's check-your-output list is effectively the marking scheme for any styling question: theme colour applied by name, component styled through its public door, and status shown with the right component.

Where the styling lives

Where the styling lives	What belongs there
<code>src/theme/variables.scss</code>	the <code>--ion-color-*</code> custom properties and the theme values
<code>src/global.scss</code>	global helpers, and the Ionic base stylesheet imports
a page's own <code>.scss</code>	styles scoped to that page

Reaching into a component

Reaching into a component	What to use
An ordinary descendant selector on an <code>ion-*</code> element	nothing: the component's Shadow DOM stops it
A part the component exposes	<code>ion-item::part(native)</code>
Changing the brand colour	edit <code>--ion-color-primary</code> , not a rule per button
A brief message at the edge of the screen	<code>ion-toast</code>

Traps

- Ordinary CSS selectors do not reach inside Ionic components. Use a component CSS variable or `::part()`.
- Do not copy internal class names out of the browser inspector. They are not public API.
- Importing the Ionic base stylesheets in `global.scss` is required before any page styling behaves as expected.
- Fixed pixel widths break the phone layout. Use the grid attributes.
- Set the colour once in `variables.scss` and refer to it by name in templates.

How the paper examines it

Lighter than L02 to L06, and almost entirely identification and short answer: the three stylesheets and their jobs, `--ion-color-primary`, the colour names and their meanings, Shadow DOM, `::part()`, and the component families.

NgModule note: No translation needed for the stylesheets, with two details: `IonicModule` must be imported by each page module for the Ionic components in that page's template to work, and in Ionic 9 it is imported from `@ionic/angular/lazy`, because the root `@ionic/angular` entry point is the standalone API. The theme files are unaffected.

Code to recognise

WHERE BRAND COLOUR LIVES, AND HOW TO REACH INSIDE A COMPONENT

```
/* Ionic Variables and Theming. */

:root {
  /* school blue + gold, Lesson 7 Example 1A */
  --ion-color-primary: #0f4c81;
  --ion-color-primary-rgb: 15, 76, 129;
  --ion-color-primary-contrast: #ffffff;
  --ion-color-primary-contrast-rgb: 255, 255, 255;
  --ion-color-primary-shade: #0d4372;
  --ion-color-primary-tint: #275e8e;

  --ion-color-secondary: #f59e0b;

  // An ordinary descendant selector cannot reach it: Shadow DOM.
  // A part the component exposes can be styled:
  ion-item::part(native) {
    border-radius: 12px;
  }
}
```

Glossary: L07

Term	What it means, as the exam states it
src/theme/variables.scss	The stylesheet holding the global theme values, including the <code>--ion-color-*</code> custom properties.
src/global.scss	The stylesheet holding global helpers and the Ionic base stylesheet imports.
<code>--ion-color-primary</code>	The CSS custom property Ionic reads to style every component that asks for the primary colour.
Shadow DOM	The encapsulation that explains why an ordinary CSS descendant selector cannot reach the inner label of an Ionic component.
<code>::part()</code>	The selector used to style an element that a component deliberately exposes, such as <code>ion-item::part(native)</code> .
ion-toast	The Ionic component that shows a brief, auto-dismissing message at the edge of the screen.
page.component.scss	Rules for one screen only.
primary	The main colour, used for the important actions.
secondary	The quieter companion colour.
tertiary	A third accent for highlighting.
success	Good, done, available.
warning	Careful.
danger	Something went wrong, or something destructive.
medium	Default, unremarkable.
light	A pale surface colour.

Project-wide facts

Not tied to one lesson. These are the mistakes that cost marks in the coded part.

1. Every `@Component` and `@NgModule` declaration needs `standalone: false` on Angular 19 and later, because `standalone` is the default.
2. Page file set is five files: `x.page.ts`, `x.page.html`, `x.page.scss`, `x-routing.module.ts`, `x.module.ts`. A page without its routing module cannot be lazily loaded.
3. `forRoot` once in `app-routing.module.ts`. `forChild` in every other routing module.
4. `loadChildren` takes a module; `loadComponent` takes a class. This project is `NgModule` based, so it is `loadChildren` throughout.
5. Asset URLs are root relative: `assets/movie-posters/iron-man.jpg`, never `../../assets/...`
6. constructor wires dependencies. `ngOnInit` is the first place page data may be touched. The decks and the class agree on this one.
7. A generated page arrives as six files: the class, the template, the page's own stylesheet, a `.page.spec.ts` stub, the routing module and the module. This project ships the five that carry behaviour and leaves the spec stub out. The module and the routing module are generator output and are never hand edited: the only part of a route a student writes is the entry inside the `routes` array.
8. In Ionic 9 the root `@ionic/angular` entry point is the standalone API and holds `provideIonicAngular()`; the class based `IonicModule` lives in `@ionic/angular/lazy`. Importing `IonicModule` from the root entry point fails the build with `NG8001` on every `ion-*` element, and `IonicModule` is deprecated in favour of `provideIonicAngular()`, which also works in `NgModule` applications.